

# **CS 5151 / 6051**

# **Database Theory (DBT)**

## **05: SQL - Intermediate**

Seokki Lee

**Slides are adapted from:**

**Database System Concepts, 6<sup>th</sup> & 7<sup>th</sup> Ed. ©Silberschatz, Korth and Sudarshan**

# Outline

- **Views**
- Transactions
- Integrity Constraints
- SQL Data Types and Schemas
- Access Control

# Views

# Views

- Any relation that is not of the conceptual model but is made visible to a user as a “virtual relation” is called a **view**.
  - Not precomputed and stored
  - Computed by executing the query whenever it is used

# Views

- Consider a person who needs to know an instructors name and department, but not the salary. This person should see a relation described, in SQL, by

```
select ID, name, dept_name  
from instructor
```

- A list of all course sections offered by the Physics department in the Fall 2017 semester with the building and room number of each section

```
select course.course id, sec id, building, room number  
from course, section  
where course.course id = section.course id  
      and course.dept_name = 'Physics'  
      and section.semester = 'Fall'  
      and section.year = 2017
```

# Views

- Possible to compute and store the results of the queries and make them available to users
  - What about the data in the relations?
  - Comparing to **with** clause

# Views

- Why are views useful?

# Views

- In some cases, it is not desirable for all users to see the entire logical model (that is, all the actual relations stored in the database.)
  - Security considerations
  - Personalized collection of “virtual” relations
- A view provides a mechanism to hide certain data from the view of certain users.



# View Definition

- A view is defined using the **create view** statement which has the form  
**create view *v* as** <query expression >
  - <query expression> is any legal SQL expression
  - *v* : the view name
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression.
  - Rather, a view definition causes the saving of an expression
  - The expression is substituted into queries using the view.

# View Definition and Use

- A view of instructors without their salary

```
create view faculty as  
    select ID, name, dept_name  
    from instructor
```

- Find all instructors in the Biology department

```
select name  
from faculty  
where dept_name = 'Biology'
```

- Create a view of department salary totals

# View Definition and Use

- A view of instructors without their salary

```
create view faculty as  
    select ID, name, dept_name  
    from instructor
```

- Find all instructors in the Biology department

```
select name  
from faculty  
where dept_name = 'Biology'
```

- Create a view of department salary totals

```
create view departments_total_salary(dept_name, total_salary) as  
    select dept_name, sum (salary)  
    from instructor  
    group by dept_name;
```

# Views Defined Using Other Views?

- One view may be used in the expression defining another view?

# Views Defined Using Other Views

- One view may be used in the expression defining another view.
- A view relation  $v_2$  is said to *depend directly* on a view relation  $v_1$  if  $v_1$  is used in the expression defining  $v_2$
- A view relation  $v_2$  is said to *depend on* view relation  $v_1$  if either  $v_2$  depends directly to  $v_1$  or there is a path of dependencies from  $v_2$  to  $v_1$
- A view relation  $v$  is said to be *recursive* if it depends on itself.

# Views Defined Using Other Views

- **create view** *physics\_fall\_2017* **as**  
    **select** *course.course\_id, sec\_id, building, room\_number*  
    **from** *course, section*  
    **where** *course.course\_id = section.course\_id*  
          **and** *course.dept\_name = 'Physics'*  
          **and** *section.semester = 'Fall'*  
          **and** *section.year = '2017'*;

# Views Defined Using Other Views

- **create view** *physics\_fall\_2017* **as**  
    **select** *course.course\_id, sec\_id, building, room\_number*  
    **from** *course, section*  
    **where** *course.course\_id = section.course\_id*  
          **and** *course.dept\_name = 'Physics'*  
          **and** *section.semester = 'Fall'*  
          **and** *section.year = '2017'*;
- **create view** *physics\_fall\_2017\_watson* **as**  
    **select** *course\_id, room\_number*  
    **from** *physics\_fall\_2017*  
    **where** *building= 'Watson'*;

# View Expansion

- Expand the view :

```
create view physics_fall_2017_watson as  
select course_id, room_number  
from physics_fall_2017  
where building= 'Watson'
```

- To:

```
create view physics_fall_2017_watson as  
select course_id, room_number  
from (select course.course_id, building, room_number  
      from course, section  
      where course.course_id = section.course_id  
           and course.dept_name = 'Physics'  
           and section.semester = 'Fall'  
           and section.year = '2017')  
where building= 'Watson';
```



# View Expansion (Cont.)

- A way to define the meaning of views defined in terms of other views.
- Let view  $v_1$  be defined by an expression  $e_1$  that may itself contain uses of view relations.
  - View expansion of an expression repeats the following replacement step:  
**repeat**  
    Find any view relation  $v_i$  in  $e_1$   
    Replace the view relation  $v_i$  by the expression defining  $v_i$   
**until** no more view relations are present in  $e_1$ 
    - As long as the view definitions are not recursive, this loop will terminate

# Materialized Views

# Materialized Views

- Certain database systems allow view relations to be physically stored.
  - Physical copy created when the view is defined.
  - Such views are called **Materialized view**
  - In which case is it useful?

# Materialized Views

- Certain database systems allow view relations to be physically stored.
  - Physical copy created when the view is defined.
  - Such views are called **Materialized view**
  - In which case is it useful?
    - Using a view frequently
    - Demanding fast response to certain queries
      - E.g., aggregates over large relations

# Materialized Views

- (Materialized) View maintenance
  - If relations used in the query are updated, the materialized view result becomes out of date.
  - Need to **maintain** the view, by updating the view whenever the underlying relations are updated
  - Lazy or Periodic

# Materialized Views

- (Materialized) View maintenance
  - If relations used in the query are updated, the materialized view result becomes out of date.
  - Need to **maintain** the view, by updating the view whenever the underlying relations are updated
  - Lazy or Periodic
    - Lazy: update the view when accessed
    - Periodic: update the view periodically
    - It should be carefully considered with storage costs and overhead for updates.
    - Some systems permit DBA to control which method to be used

# Update of a View

- Useful but serious problems may occur with modification
  - Updates, insertions, or deletions
- Add a new tuple to *faculty* view which we defined earlier  
**insert into *faculty* values** ('30765', 'Green', 'Music');

# Update of a View

- Useful but serious problems may occur with modification
  - Updates, insertions, or deletions
- Add a new tuple to *faculty* view which we defined earlier  
**insert into *faculty* values** ('30765', 'Green', 'Music');
- This insertion must be represented by the insertion into the *instructor* relation.
  - instructor(ID, name, dept name, salary)
  - Should have a value for salary



# Update of a View

- Two approaches
  - Reject the insert
  - Insert the tuple  
('30765', 'Green', 'Music', null)  
into the *instructor* relation

# Some Updates Cannot be Translated Uniquely

- **create view** *instructor\_info* **as**  
    **select** *ID, name, building*  
    **from** *instructor, department*  
    **where** *instructor.dept\_name= department.dept\_name;*
- **insert into** *instructor\_info* **values** ('69987', 'White', 'Taylor');

*instructor*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 12121     | Wu          | Finance          | 90000         |
| 15151     | Mozart      | Music            | 40000         |
| 22222     | Einstein    | Physics          | 95000         |
| 32343     | El Said     | History          | 60000         |
| 33456     | Gold        | Physics          | 87000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 58583     | Califieri   | History          | 62000         |
| 76543     | Singh       | Finance          | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |

*department*

| <i>dept_name</i> | <i>building</i> | <i>budget</i> |
|------------------|-----------------|---------------|
| Biology          | Watson          | 90000         |
| Comp. Sci.       | Taylor          | 100000        |
| Electrical Eng.  | Taylor          | 85000         |
| Finance          | Painter         | 120000        |
| History          | Painter         | 50000         |
| Music            | Packard         | 80000         |
| Physics          | Watson          | 70000         |

# Some Updates Cannot be Translated Uniquely

- **create view** *instructor\_info* **as**  
    **select** *ID, name, building*  
    **from** *instructor, department*  
    **where** *instructor.dept\_name= department.dept\_name;*
- **insert into** *instructor\_info* **values** ('69987', 'White', 'Taylor');

*instructor*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 12121     | Wu          | Finance          | 90000         |
| 15151     | Mozart      | Music            | 40000         |
| 22222     | Einstein    | Physics          | 95000         |
| 32343     | El Said     | History          | 60000         |
| 33456     | Gold        | Physics          | 87000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 58583     | Califieri   | History          | 62000         |
| 76543     | Singh       | Finance          | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 69987     | White       | <i>null</i>      | <i>null</i>   |

*department*

| <i>dept_name</i> | <i>building</i> | <i>budget</i> |
|------------------|-----------------|---------------|
| Biology          | Watson          | 90000         |
| Comp. Sci.       | Taylor          | 100000        |
| Electrical Eng.  | Taylor          | 85000         |
| Finance          | Painter         | 120000        |
| History          | Painter         | 50000         |
| Music            | Packard         | 80000         |
| Physics          | Watson          | 70000         |
| <i>null</i>      | Taylor          | <i>null</i>   |

# Some Updates Cannot be Translated Uniquely

- Any problem?

instructor

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 12121     | Wu          | Finance          | 90000         |
| 15151     | Mozart      | Music            | 40000         |
| 22222     | Einstein    | Physics          | 95000         |
| 32343     | El Said     | History          | 60000         |
| 33456     | Gold        | Physics          | 87000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 58583     | Califieri   | History          | 62000         |
| 76543     | Singh       | Finance          | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 69987     | White       | <i>null</i>      | <i>null</i>   |

department

| <i>dept_name</i> | <i>building</i> | <i>budget</i> |
|------------------|-----------------|---------------|
| Biology          | Watson          | 90000         |
| Comp. Sci.       | Taylor          | 100000        |
| Electrical Eng.  | Taylor          | 85000         |
| Finance          | Painter         | 120000        |
| History          | Painter         | 50000         |
| Music            | Packard         | 80000         |
| Physics          | Watson          | 70000         |
| <i>null</i>      | Taylor          | <i>null</i>   |

# Some Updates Cannot be Translated Uniquely

- Which department, if multiple departments in Taylor?
  - Unknown department is at Taylor
- What if no department is in Taylor?
  - Violation on primary key constraint
  - ('69987', 'White', 'Taylor') will not appear in the view.

instructor

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 12121     | Wu          | Finance          | 90000         |
| 15151     | Mozart      | Music            | 40000         |
| 22222     | Einstein    | Physics          | 95000         |
| 32343     | El Said     | History          | 60000         |
| 33456     | Gold        | Physics          | 87000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 58583     | Califieri   | History          | 62000         |
| 76543     | Singh       | Finance          | 80000         |
| 76766     | Crick       | Biology          | 72000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 69987     | White       | <i>null</i>      | <i>null</i>   |

department

| <i>dept_name</i> | <i>building</i> | <i>budget</i> |
|------------------|-----------------|---------------|
| Biology          | Watson          | 90000         |
| Comp. Sci.       | Taylor          | 100000        |
| Electrical Eng.  | Taylor          | 85000         |
| Finance          | Painter         | 120000        |
| History          | Painter         | 50000         |
| Music            | Packard         | 80000         |
| Physics          | Watson          | 70000         |
| <i>null</i>      | Taylor          | <i>null</i>   |

# Some Updates Cannot be Translated Uniquely

- In general, modifications are not permitted on view relations.
- Most SQL implementations allow updates only on simple views.
  - The **from** clause has only **one** database **relation**.
  - The **select** clause contains **only attribute names** of the relation, and does not have any expressions, aggregates, or **distinct** specification.
  - Any attribute not listed in the **select** clause can be set to null.
  - The query does not have a **group** by or **having** clause.
- Materialized views are not updatable.

# And Some Not at All

- **create view** *history\_instructors* **as** (  
    **select** \*  
    **from** *instructor*  
    **where** *dept\_name*= 'History');
- What happens if we insert  
    ('25566', 'Brown', 'Biology', 100000)  
into *history\_instructors*?

# And Some Not at All

- **create view** *history\_instructors* **as** (  
    **select** \*  
    **from** *instructor*  
    **where** *dept\_name*= 'History')  
**with check option**;
- What happens if we insert  
    ('25566', 'Brown', 'Biology', 100000)  
    into *history\_instructors*?
- By default, it would be allowed.
  - Views can be defined with a **with check option**.
  - If the inserted tuple doesn't satisfy the condition, the insertion is rejected.
  - Updates are similarly rejected.



# Outline

- Views
- **Transactions**
- Integrity Constraints
- SQL Data Types and Schemas
- Access Control

# Transactions

- Unit of work
  - Consists of a sequence of query and/or update statements

# Transactions

- Unit of work
  - Consists of a sequence of query and/or update statements
- Atomic transaction
  - Either fully executed or rolled back as if it never occurred
  - Isolation from concurrent transactions

# Transactions

- Unit of work
  - Consists of a sequence of query and/or update statements
- Atomic transaction
  - Either fully executed or rolled back as if it never occurred
  - Isolation from concurrent transactions
- Transactions **begin** implicitly when an SQL statement is executed
  - Ended by **commit (work)** or **rollback (work)**
- But default on most databases: each SQL statement commits automatically
  - Can turn off auto commit for a session (e.g. using API)

# Transactions Example

- Atomicity (all-or-nothing)
  - Relation **accounts(accID, cust, type, balance)**
  - A customer wants to transfer \$100 from his savings (accID = 100) to his checking account (accID= 101)  
**UPDATE** accounts **SET** balance = balance – 100 **WHERE** accID = 100;  
**UPDATE** accounts **SET** balance = balance + 100 **WHERE** accID = 101;

# Transactions Example

- Atomicity (all-or-nothing)
  - Relation **accounts(accID, cust, type, balance)**
  - A customer wants to transfer \$100 from his savings (accID = 100) to his checking account (accID= 101)  
**UPDATE** accounts **SET** balance = balance – 100 **WHERE** accID = 100;  
**UPDATE** accounts **SET** balance = balance + 100 **WHERE** accID = 101;
- This can cause inconsistencies if the system crashes after the first update (user would loose money)
  - Using a transaction either both or none of the statements are executed  
**BEGIN**  
**UPDATE** accounts **SET** balance = balance – 100 **WHERE** accID = 100;  
**UPDATE** accounts **SET** balance = balance + 100 **WHERE** accID = 101;  
**COMMIT**

# Transactions and Concurrency

- Also used to isolate concurrent actions of different users
  - Recall that if several users are modifying the database at the same time that can lead to inconsistencies

# Outline

- Views
- Transactions
- **Integrity Constraints**
- SQL Data Types and Schemas
- Access Control



# Integrity Constraints

# Integrity Constraints

- Integrity constraints **guard against accidental** damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency.
  - A checking account must have a balance greater than \$10,000.
  - A salary of a bank employee must be at least \$40.00 an hour.
  - A customer must have a (non-null) phone number.
- Usually defined as part of the database schema design process
  - Can be also added to an existing relation

# Constraints on a Single Relation

- **not null**
- **primary key**
- **unique**
- **check** (P), where P is a predicate

# Not Null Constraints

- not null

# Not Null Constraints

- **not null**
  - Prohibits the insertion of a null value for the attribute
  - Declare *name* and *budget* to be **not null**  
*name* **varchar(20) not null**  
*budget* **numeric(12,2) not null**

# Unique Constraints

- **unique** ( $A_1, A_2, \dots, A_m$ )

# Unique Constraints

- **unique** ( $A_1, A_2, \dots, A_m$ )
  - The unique specification states that the attributes  $A_1, A_2, \dots, A_m$  form a superkey.
  - Null permitted?

# Unique Constraints

- **unique** ( $A_1, A_2, \dots, A_m$ )
  - The unique specification states that the attributes  $A_1, A_2, \dots, A_m$  form a superkey.
  - The keys are permitted to be null (in contrast to primary keys).



# The Check Clause

- The **check**(P) clause specifies a predicate P that must be satisfied by every tuple in a relation.
  - Permits attribute domains to be restricted in powerful ways

# The Check Clause

- The **check**(P) clause specifies a predicate P that must be satisfied by every tuple in a relation.
  - Permits attribute domains to be restricted in powerful ways
- Example: ensure that semester is one of fall, winter, spring or summer

```
create table section
  (course_id varchar (8),
   sec_id varchar (8),
   semester varchar (6),
   year numeric (4,0),
   building varchar (15),
   room_number varchar (7),
   time slot id varchar (4),
   primary key (course_id, sec_id, semester, year),
   check (semester in ('Fall', 'Winter', 'Spring', 'Summer')))
```

# The Check Clause

- Example: ensure that semester is one of fall, winter, spring or summer

```
create table section
  (course_id varchar (8),
   sec_id varchar (8),
   semester varchar (6),
   year numeric (4,0),
   building varchar (15),
   room_number varchar (7),
   time slot id varchar (4),
   primary key (course_id, sec_id, semester, year),
   check (semester in ('Fall', 'Winter', 'Spring', 'Summer')))
```

- What about semester has null values?

# The Check Clause

- Example: ensure that semester is one of fall, winter, spring or summer

```
create table section
  (course_id varchar (8),
   sec_id varchar (8),
   semester varchar (6),
   year numeric (4,0),
   building varchar (15),
   room_number varchar (7),
   time slot id varchar (4),
   primary key (course_id, sec_id, semester, year),
   check (semester in ('Fall', 'Winter', 'Spring', 'Summer')))
```

- What about semester has null values?
  - A check clause is satisfied if not false.

# Complex Check Conditions

- The predicate in the check clause can be an arbitrary predicate that can include a **subquery**.

**create table** *section*

```
(course_id varchar (8),  
  sec_id varchar (8),  
  semester varchar (6),  
  year numeric (4,0),  
  building varchar (15),  
  room_number varchar (7),  
  time_slot_id varchar (4),  
  primary key (course_id, sec_id, semester, year),  
  check (semester in ('Fall', 'Winter', 'Spring', 'Summer')),  
  check (time_slot_id in (select time_slot_id from time_slot))
```

| <i>section</i>          |
|-------------------------|
| <u><i>course_id</i></u> |
| <u><i>sec_id</i></u>    |
| <u><i>semester</i></u>  |
| <u><i>year</i></u>      |
| <i>building</i>         |
| <i>room_no</i>          |
| <i>time_slot_id</i>     |

| <i>time_slot</i>           |
|----------------------------|
| <u><i>time_slot_id</i></u> |
| <u><i>day</i></u>          |
| <u><i>start_time</i></u>   |
| <u><i>end_time</i></u>     |

# Complex Check Conditions

- The predicate in the check clause can be an arbitrary predicate that can include a **subquery**.

**check** (*time\_slot\_id* **in** (**select** *time\_slot\_id* **from** *time\_slot*))

- The check condition states that the *time\_slot\_id* in each tuple in the *section* relation is actually the identifier of a time slot in the *time\_slot* relation.
- The condition has to be checked not only when a tuple is inserted or modified in *section*, but also when the relation *time\_slot* changes

# Referential Integrity

- Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
  - Example: If “Biology” is a department name appearing in one of the tuples in the ***instructor*** relation, then there exists a tuple in the ***department*** relation for “Biology”.
- Let A be a set of attributes. Let R and S be two relations that contain attributes A and where A is the primary key of S. A is said to be a **foreign key** of R if for any values of A appearing in R these values also appear in S.

# Referential Integrity (Cont.)

- Foreign keys can be specified as part of the SQL **create table** statement  
**foreign key** (*dept\_name*) **references** *department*
- By default, a foreign key *references* the primary-key attributes of the *referenced* table.
- SQL allows a list of attributes of the referenced relation to be specified explicitly.  
**foreign key** (*dept\_name*) **references** *department* (*dept\_name*)



# Cascading Actions in Referential Integrity

- When a referential-integrity constraint is **violated**, the normal procedure is to reject the action that caused the violation.

- An alternative, in case of delete or update is to cascade

```
create table course (  
    (...  
    dept_name varchar(20),  
    foreign key (dept_name) references department  
        on delete cascade  
        on update cascade,  
    . . .)
```

- Instead of cascade we can use :
  - **set null** (set the field to null)
  - **set default** (set the field to the default value for the domain)

# When to Verify Integrity Constraint Violation

- After a single database modification
  - Insert, update or delete
- At the end of a transaction
  - After a sequence of modifications

# Integrity Constraint Violation During Transactions

- Consider:

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```

- How to insert a person who does not have father/mother without causing constraint violation?

# Integrity Constraint Violation During Transactions

- Consider:

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```

- How to insert a person who does not have father/mother without causing constraint violation?
  - Insert father and mother of a person before inserting person

# Integrity Constraint Violation During Transactions

- Consider:

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```

- How to insert a person who does not have father/mother without causing constraint violation?
  - Insert father and mother of a person before inserting person
  - Set father and mother to null initially, update after inserting all persons
    - not possible if father and mother attributes declared to be **not null**

# Integrity Constraint Violation During Transactions

- Consider:

```
create table person (  
    ID char(10),  
    name char(40),  
    mother char(10),  
    father char(10),  
    primary key ID,  
    foreign key father references person,  
    foreign key mother references person)
```

- How to insert a person who does not have father/mother without causing constraint violation?
  - Insert father and mother of a person before inserting person
  - Set father and mother to null initially, update after inserting all persons
    - not possible if father and mother attributes declared to be **not null**
  - Defer constraint checking

# Assertions

- An **assertion** is a predicate expressing a condition that we wish the database always to satisfy.
- The following constraints, can be expressed using assertions:
  - For each tuple in the *student* relation, the value of the attribute *tot\_cred* must equal the sum of credits of courses that the student has completed successfully.
  - An instructor cannot teach in two different classrooms in a semester in the same time slot.

# Assertions

- An assertion in SQL takes the form:

**create assertion** <assertion-name> **check** (<predicate>);

- For each tuple in the student relation, the value of the attribute tot\_cred must equal the sum of credits of courses that the student has completed successfully.

```
create assertion credits_earned_constraint check  
(not exists (select ID  
             from student  
             where tot_cred <> (select coalesce(sum(credits), 0)  
                                from takes natural join course  
                                where student.ID= takes.ID  
                                and grade is not null and grade<> 'F' )))
```



# Outline

- Views
- Transactions
- Integrity Constraints
- **SQL Data Types and Schemas**
- Access Control

# Built-in Data Types in SQL

- **date**: Calendar dates, containing a (4 digit) year, month and date
  - Example: **date** '2005-7-27'
- **time**: Time of day, in hours, minutes and seconds.
  - Example: **time** '09:00:30'      **time** '09:00:30.75'
- **timestamp**: date plus time of day
  - Used to specify the number of fractional digits for seconds
  - Example: **timestamp** '2005-7-27 09:00:30.75'
- **interval**: period of time
  - Example: interval '1' day
  - x and y are **date** → x - y is an interval (number of days from x to y)
  - Adding or subtracting an interval from a date/time gives back a date/time, respectively.

# Large-Object Types

- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*:
  - **blob**: binary large object -- object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system)
  - **clob**: character large object -- object is a large collection of character data

*book\_review* **clob**(10KB)

*image* **blob**(10MB)

*movie* **blob**(2GB)

# Large-Object Types

- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*:
  - **blob**: binary large object -- object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system)
  - **clob**: character large object -- object is a large collection of character data

*book\_review* **clob**(10KB)

*image* **blob**(10MB)

*movie* **blob**(2GB)

- When a query returns a large object, a pointer is returned rather than the large object itself.

# User-Defined Types

- **create type** construct in SQL creates user-defined type

**create type** *Dollars* **as numeric (12,2)**

- Example:

```
create table department  
(dept_name varchar (20),  
  building varchar (15),  
  budget Dollars);
```

# Domains

- **create domain** construct in SQL-92 creates user-defined domain types

**create domain** *person\_name* **char**(20) **not null**

- Types and domains are similar. Domains can have constraints, such as **not null**, specified on them.
- Example:

**create domain** *degree\_level* **varchar**(10)  
**constraint** *degree\_level\_test*  
**check** (**value in** ('Bachelors', 'Masters', 'Doctorate'));

# Index Creation

# Index Creation

- Many queries reference only a small proportion of the records in a table.
  - Find the salary value of the instructor with ID 22201.
- It is inefficient for the system to read every record to find a record with particular value.



# Index Creation

- Many queries reference only a small proportion of the records in a table.
  - Find the salary value of the instructor with ID 22201.
- It is inefficient for the system to read every record to find a record with particular value.
- An **index** on an attribute of a relation is a data structure that allows the database system to find those tuples in the relation that have a specified value for that attribute efficiently, **without scanning through all the tuples** of the relation.
- We create an index with the **create index** command.  
**create index** <name> **on** <relation-name> (attribute);

# Index Creation Example

- **create table** *student* (

*ID*                      **varchar** (5),  
*name*                   **varchar** (20) **not null**,  
*dept\_name*           **varchar** (20),  
*tot\_cred*              **numeric** (3,0) **default** 0,  
**primary key** (*ID*))

- **create index** *studentID\_index* **on** *student*(*ID*)

- The query:

**select** \*  
**from** *student*  
**where** *ID* = '12345'

can be executed by using the index to find the required record, without looking at all records of *student*

# Outline

- Views
- Transactions
- Integrity Constraints
- SQL Data Types and Schemas
- **Access Control**

# Authorization

- We may assign a user several forms of authorizations on parts of the database.
  - **Read** - allows reading, but not modification of data.
  - **Insert** - allows insertion of new data, but not modification of existing data.
  - **Update** - allows modification, but not deletion of data.
  - **Delete** - allows deletion of data.
- Each of these types of authorizations is called a **privilege**.
  - We may authorize the user all, none, or a combination of these types of privileges on specified parts of a database, such as a relation or a view.
- When a user submits a query or an update, the SQL implementation first checks if they are authorized.

# Authorization (Cont.)

- Forms of authorization to modify the database schema
  - **Index** - allows creation and deletion of indices.
  - **Resources** - allows creation of new relations.
  - **Alteration** - allows addition or deletion of attributes in a relation.
  - **Drop** - allows deletion of relations.

# Authorization Specification in SQL

- The **grant** statement is used to confer authorization.  
**grant** <privilege list> **on** <relation or view > **to** <user list>
- <privilege list>: **select**, **insert**, **update**, and **delete**
- <user list>
  - a user-id
  - **public**, which allows all valid users the privilege granted
  - A role (more on this later)

# Privileges in SQL

- **select**: allows read access to relation, or the ability to query using the view
  - Example: grant users  $U_1$ ,  $U_2$ , and  $U_3$  **select** authorization on the *instructor* relation:

**grant select on *instructor* to  $U_1$ ,  $U_2$ ,  $U_3$**

- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **all privileges**: used as a short form for all the allowable privileges

# Authorization Specification in SQL

- Example:
  - **grant select on *department* to** Amit, Satoshi
  - **grant update(budget) on *department* to** Amit, Satoshi
- Granting a privilege on a view **does not imply** granting any privileges on the underlying relations.
- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).



# Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization.  
**revoke** <privilege list> **on** <relation or view> **from** <user list>
- Example:  
**revoke select on** *student* **from**  $U_1, U_2, U_3$   
**revoke select on** department **from** Amit, Satoshi;  
**revoke update** (budget) **on** department **from** Amit, Satoshi;

# Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization.  
**revoke** <privilege list> **on** <relation or view> **from** <user list>
  - <privilege list> may be **all** to revoke all privileges the revokee may hold.
  - If <revokee list> includes **public**, all users lose the privilege except those granted it explicitly.
  - If the same privilege was granted twice to the same user by different grantees, the user may retain the privilege after the revocation.
  - All privileges that **depend on** the privilege being revoked are also **revoked**.

# Roles

- A **role** is a way to distinguish among various users as far as what these users can access/update in the database.
- To create a role we use: **create role** <name>
- Example:
  - **create role** instructor
- Once a role is created we can assign “users” to the role using:
  - **grant** <role> **to** <users>
- What about creating an instructor user\_id (permitting each instructor to use the id to connect to the database)?

# Roles

- A **role** is a way to distinguish among various users as far as what these users can access/update in the database.
- To create a role we use: **create role** <name>
- Example:
  - **create role** instructor
- Once a role is created we can assign “users” to the role using:
  - **grant** <role> **to** <users>
- What about creating an instructor user\_id (permitting each instructor to use the id to connect to the database)?
  - Not possible to identify which instructor carried out an update
  - If left or moved → need to create a new password and distributed it in a secure manner

# Roles Example

- **create role** instructor;
- **grant** *instructor* **to** Amit;
- Privileges can be granted to roles:
  - **grant select on** *takes* **to** *instructor*;
- Roles can be granted to users, as well as to other roles
  - **create role** *teaching\_assistant*
  - **grant** *teaching\_assistant* **to** *instructor*;
    - *Instructor* inherits all privileges of *teaching\_assistant*.
- Chain of roles
  - **create role** *dean*;
  - **grant** *instructor* **to** *dean*;
  - **grant** *dean* **to** Satoshi;

# Authorization on Views

- Consider a staff member who needs to know the salaries of all faculty in Geology department.
  - Not authorized to see them in other departments

# Authorization on Views

- Consider a staff member who needs to know the salaries of all faculty in Geology department.
  - Not authorized to see them in other departments
- **create view** *geo\_instructor* **as**  
(**select** \*  
  **from** *instructor*  
  **where** *dept\_name* = 'Geology');
- **grant select on** *geo\_instructor* **to** *geo\_staff*
- Suppose that a *geo\_staff* member issues
  - **select** \*  
  **from** *geo\_instructor*;

# Authorization on Views

- **create view** *geo\_instructor* **as**  
(**select** \*  
  **from** *instructor*  
  **where** *dept\_name* = 'Geology');
- **grant select on** *geo\_instructor* **to** *geo\_staff*
- Suppose that a *geo\_staff* member issues
  - **select** \*  
  **from** *geo\_instructor*;
- What if
  - *geo\_staff* does not have permissions on *instructor*?



# Authorization on Views

- **create view** *geo\_instructor* **as**  
(**select** \*  
  **from** *instructor*  
  **where** *dept\_name* = 'Geology');
- **grant select on** *geo\_instructor* **to** *geo\_staff*
- Suppose that a *geo\_staff* member issues
  - **select** \*  
  **from** *geo\_instructor*;
- What if
  - *geo\_staff* does not have permissions on *instructor*?
    - The staff must not have direct access to the instructor table.
    - The result of the query is visible to the staff.

# Authorization on Views

- **create view** *geo\_instructor* **as**  
(**select** \*  
  **from** *instructor*  
  **where** *dept\_name* = 'Geology');
- **grant select on** *geo\_instructor* **to** *geo\_staff*
- Suppose that a *geo\_staff* member issues
  - **select** \*  
  **from** *geo\_instructor*;
- What if
  - *geo\_staff* does not have permissions on *instructor*?
    - The staff must not have direct access to the instructor table.
    - The result of the query is visible to the staff.
  - Creator of view did not have some permissions on *instructor*?

# Authorization on Views

- **create view** *geo\_instructor* **as**  
(**select** \*  
  **from** *instructor*  
  **where** *dept\_name* = 'Geology');
- **grant select on** *geo\_instructor* **to** *geo\_staff*
- Suppose that a *geo\_staff* member issues
  - **select** \*  
  **from** *geo\_instructor*;
- What if
  - *geo\_staff* does not have permissions on *instructor*?
    - The staff must not have direct access to the instructor table.
    - The result of the query is visible to the staff.
  - Creator of view did not have some permissions on *instructor*?
    - The creator must have **select** authorization on the *instructor* relation.

# Authorizations on Schema

- SQL standard specifies a primitive authorization mechanism for the database schema.
  - Only the owner of the schema can carry out any modification.
- **references** privilege permits to create foreign key.
  - **grant reference** (*dept\_name*) **on** *department* **to** Mariano;

# Authorizations on Schema

- SQL standard specifies a primitive authorization mechanism for the database schema.
  - Only the owner of the schema can carry out any modification.
- **references** privilege permits to create foreign key.
  - **grant reference** (*dept\_name*) **on** *department* **to** Mariano;
  - Why does Mariano need privilege on department?

# Authorizations on Schema

- SQL standard specifies a primitive authorization mechanism for the database schema.
  - Only the owner of the schema can carry out any modification.
- **references** privilege permits to create foreign key.
  - **grant reference** (*dept\_name*) **on** *department* **to** Mariano;
  - Why does Mariano need privilege on *department*?
    - Suppose Mariano creates a foreign key on *dept\_name* in table *r*.
      - Insert a tuple with Geology department into *r*
    - He should have an authorization on *dept\_name* of *department*.
    - It is no longer possible to delete the Geology department from the *department* relation without modifying the table *r*.
  - It is also required to create **check** constraint referencing *department* on *r*.

# Other Authorization Features

- A user granted may be allowed to pass on the authorizations to others.
- Transfer of privileges
  - **grant select on *department* to Amit with grant option;**
  - **revoke select on *department* from Amit, Satoshi cascade;**
  - **revoke select on *department* from Amit, Satoshi restrict;**
  - And more!

# Corresponding Reading Materials

- Database System Concepts 6<sup>th</sup> & 7<sup>th</sup> Edition
  - Chapter 4